Society for Computer Technology & Research's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, Pune.

DEPARTMENT OF INFORMATION TECHNOLOGY

DSA MINI PROJECT - CASE STUDY-BASED DATA STRUCTURES ACTIVITY

GROUP REPORT

Class: SY Division: 10 Group No.: 8

Day & Date of Presentation: Wednesday, 28/10/2026

Group members Name and Roll No.:

1. Arnav Sankapal (23206)

2. Arnav Patidar (23208)

3. Ayaan Shaikh (23210)

4. Charu Singla (23215)

Topic/Case Study Title: Canteen Order, Token, Billing and Preparation Management System

Problem statement:

A college canteen serves many orders at once. Each order needs a unique token, an itemised bill, and a fair service order in which priority customers are served first, while several orders are prepared in parallel. This project builds a menu-driven C++ system that issues tokens and bills, schedules orders across four serving stations, tracks status automatically, and saves orders and menu prices for the manager.

Objectives:

1. Automate order placement with a unique token (starting at 101), a bill number (starting at BILL-5001) and an itemised bill, with a 10% surcharge for priority orders.

2. Serve orders fairly using separate priority and normal queues feeding four serving stations, with fast token lookup, status tracking and cancellation of pending orders.

3. Keep orders and menu prices in text files and give the manager reports: all orders with stations and queues, a daily report, price updates, and completed orders sorted by bill amount.

Data structures used:

1. **Linked-list FIFO queues (priority and normal):** keep arrival order within each class; enqueue and dequeue are O(1); the priority queue is served first; a cancelled token can be removed from the middle.

2. **Hash table with separate chaining (211 buckets, token % 211):** finds an order from its token in O(1) average time instead of scanning all orders.

3. **Array of four ServingStation objects:** a fixed number of stations, so up to four orders can be prepared at the same time.

4. **Vectors (order register and menu):** the register is a vector of unique_ptr<Order>, so records stay in insertion order and the hash table has stable pointers to each order.

5. **Linear search and insertion sort:** bill-number lookup, and sorting completed orders by bill amount (small daily list).

6. **Sequential text files:** orders.txt and menu_prices.txt give readable storage and restart recovery.

Implementation (Programming language / tools):

C++17 (object-oriented: MenuItem, OrderLine, Order, Bill, LinkedOrderQueue, OrderHashTable, ServingStation, CanteenSystem). STL containers, and std::thread with std::mutex for the background timer that updates order status every second. The menu has 49 items in five categories (Breakfast, Snacks, Meals, Drinks, Desserts). Compiled with g++ -std=c++17 -pthread; menu-driven console application.

OOP concepts used:

- **Encapsulation:** private data members with public getters; only the friend class CanteenSystem changes an Order.
- **Abstraction:** CanteenSystem exposes methods such as placeOrder(), cancelPendingOrder() and findOrderByToken(); Bill::print() hides bill formatting.
- **Composition:** CanteenSystem contains the menu, order register, hash table, two queues and the station array.
- **Constructors, destructors and copy control:** constructors set up Order and MenuItem; destructors free queue and hash-table nodes and stop the timer thread; copying is deleted for the queues, hash table and CanteenSystem.

Key features of the system:

- **Token and bill generation:** unique token and bill number for every order, with an itemised bill showing item ID, quantity, rate, amount, food subtotal, priority charge and total payable. A bill preview is shown before the customer confirms.
- **Priority service:** priority orders pay 10% extra on the food subtotal and are taken from the priority queue before the normal queue; orders inside each queue are served in arrival order.
- **Customer mode:** view the menu, place a new order, check status by token, cancel a pending order, and generate or reprint a bill by order ID or bill number.
- **Manager mode:** display all orders, stations and queues; daily report; view menu and pricing; update an item price (old bills keep their original prices); sort completed orders by bill amount.
- **Order states:** PENDING, PREPARING, COMPLETED and CANCELLED. A timer thread, protected by a mutex, assigns free stations and completes orders when their preparation time is over.
- **Persistence:** orders.txt and menu_prices.txt are saved after every change and loaded again on restart.
- **Scope:** the system is an educational simulation; payment and refunds are not modelled.

Order processing workflow:

1. The customer chooses items and quantities, selects normal or priority service, checks the bill preview and confirms.
2. The system issues the next token and bill number, calculates the bill and preparation time, stores the order in the register and inserts its token into the hash table.
3. The token is added to the priority queue or the normal queue.

4. Whenever a station is free, the next order is dequeued (priority queue first), the station is assigned and the start time and expected ready time are recorded.
5. The timer thread marks the order COMPLETED when the expected ready time is reached, frees the station and starts the next waiting order.
6. The customer can check status or reprint the bill; the manager can view reports and the sorted list of completed orders.

Time complexity of the main operations:

Operation	Data structure	Time
Add order to queue (enqueue)	Linked-list FIFO queue	O(1)
Take next order (dequeue)	Linked-list FIFO queue	O(1)
Cancel a pending order	Linked-list FIFO queue (removeToken)	O(n)
Find order by token	Hash table, separate chaining	O(1) average
Find order by bill number	Vector, linear search	O(n)
Sort completed orders by bill amount	Insertion sort	$O(n^2)$

Data files and record format:

- **orders.txt:** one header line per order (token, bill number, priority flag, status code, station, order time, start time, expected ready time, completed time, subtotal, priority charge, total, preparation minutes, item count) followed by one line per item (item ID, quantity, unit price, preparation minutes, item name, category). See Fig. 4.
- **menu_prices.txt:** one line per menu item with its ID and current price, so price changes made by the manager remain after the program is closed.

Contribution of each group member:

Member	Contribution
Arnav Sankapal (23206)	Object-oriented design and implementation of the data structures: classes MenuItem, OrderLine, Order, Bill, ServingStation and the structure of CanteenSystem; LinkedOrderQueue and OrderHashTable.
Arnav Patidar (23208)	Station scheduling and concurrency: priority-first selection of the next order, assignment of the four stations, and the timer thread with mutex for automatic status updates.
Ayaan Shaikh (23210)	Customer mode: menu display, order placement with bill preview, bill printing and reprint, status check, cancellation and input validation.
Charu Singla (23215)	File handling and manager mode: saving and loading orders.txt and menu_prices.txt, daily report, price update, insertion-sort listing of completed orders, testing and report preparation.

Implementation output:

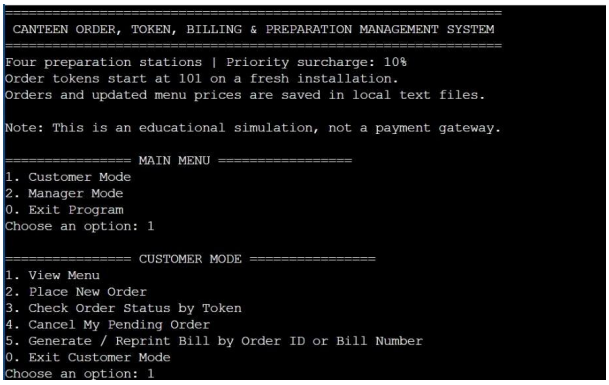


Fig. 1: Main menu

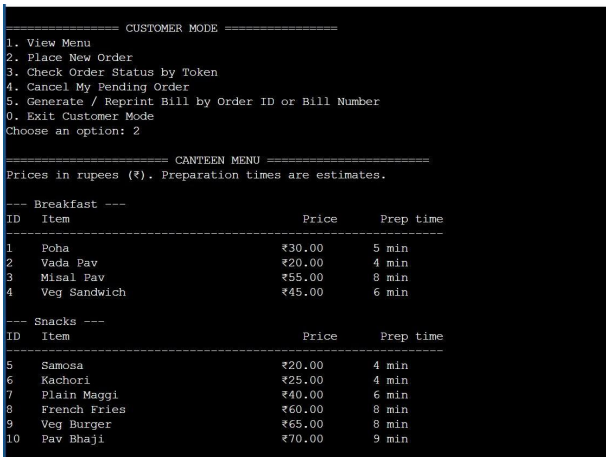


Fig. 2: Customer menu with prices and preparation times

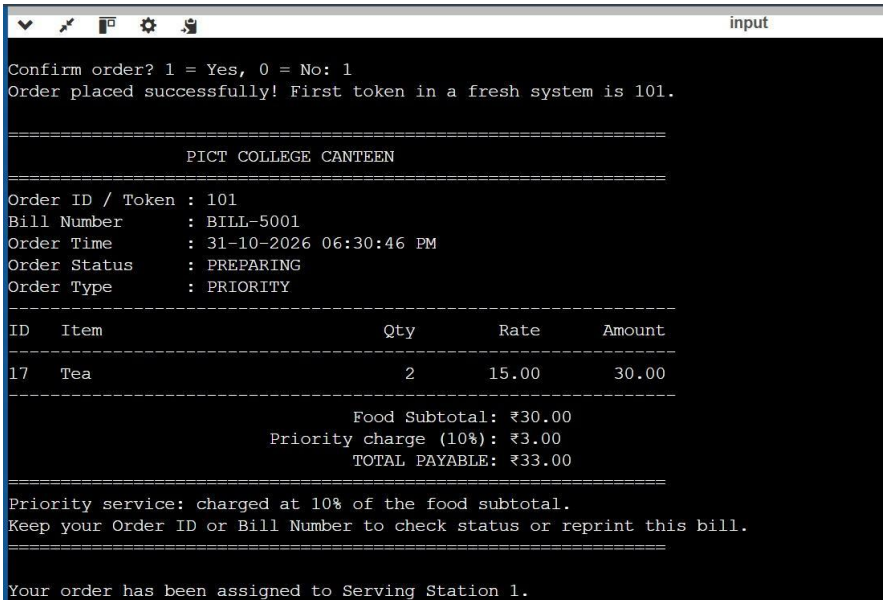


Fig. 3: Order confirmation and itemised bill

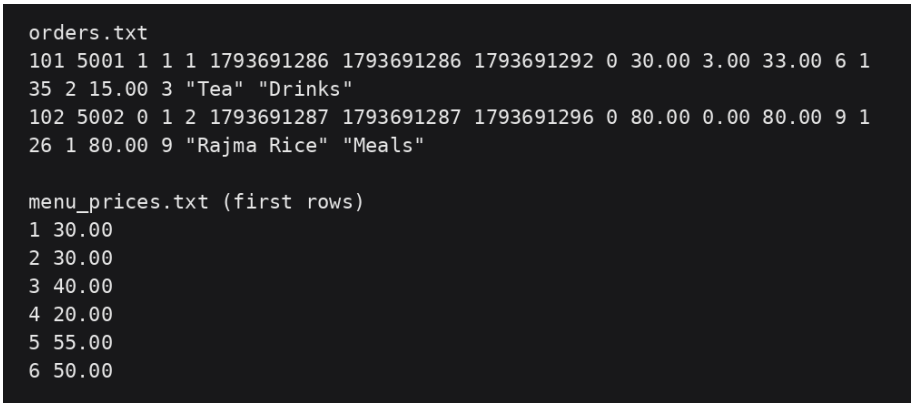


Fig. 4: Saved records (orders.txt and menu_prices.txt)

```
================ FOUR SERVING STATIONS ================
Station 1: Token 101 | NORMAL | expected ready 03-11-2026 07:35:49 AM
Station 2: Token 102 | NORMAL | expected ready 03-11-2026 07:35:50 AM
Station 3: Token 103 | NORMAL | expected ready 03-11-2026 07:35:51 AM
Station 4: Token 104 | NORMAL | expected ready 03-11-2026 07:35:51 AM

================ WAITING QUEUES ======================
Priority queue (FIFO): 106
Normal queue (FIFO)  : 107

================ PENDING ORDERS ================
Token 106 | BILL-5006 | PRIORITY | ₹27.50 | placed 03-11-2026 07:34:53 AM
Token 107 | BILL-5007 | NORMAL | ₹25.00 | placed 03-11-2026 07:34:54 AM

================ PREPARING ORDERS ================
Token 101 | BILL-5001 | NORMAL | ₹550.00 | placed 03-11-2026 07:34:49 AM | station 1
Token 102 | BILL-5002 | NORMAL | ₹550.00 | placed 03-11-2026 07:34:50 AM | station 2
Token 103 | BILL-5003 | NORMAL | ₹550.00 | placed 03-11-2026 07:34:51 AM | station 3
Token 104 | BILL-5004 | NORMAL | ₹550.00 | placed 03-11-2026 07:34:51 AM | station 4

================ COMPLETED ORDERS ================
No orders in this section.

================ CANCELLED ORDERS ================
Token 105 | BILL-5005 | NORMAL | ₹20.00 | placed 03-11-2026 07:34:52 AM
```

Fig. 5: Manager view of stations, priority and normal queues and a cancelled order

```
================ FOUR SERVING STATIONS ================
Station 1: Token 106 | PRIORITY | expected ready 03-11-2026 07:36:50 AM
Station 2: Token 102 | NORMAL | expected ready 03-11-2026 07:36:47 AM
Station 3: Token 103 | NORMAL | expected ready 03-11-2026 07:36:48 AM
Station 4: Token 104 | NORMAL | expected ready 03-11-2026 07:36:48 AM

================ WAITING QUEUES ======================
Priority queue (FIFO): Empty
Normal queue (FIFO)  : 105
```

Fig. 6: Priority token 106 gets the freed station before earlier normal token 105

```
======== COMPLETED ORDERS SORTED BY BILL AMOUNT (ASCENDING) ========
Order ID    Bill Number     Order Type       Bill Amount
-------------------------------------------------------
102         BILL-5002       NORMAL             ₹20.00
103         BILL-5003       PRIORITY           ₹44.00
101         BILL-5001       NORMAL             ₹75.00
Sorting algorithm: Insertion Sort | Time complexity: O(n^2)
```

Fig. 7: Completed orders sorted by bill amount using insertion sort

Observations from the sample runs:

- Fig. 3: token 101 (BILL-5001) is a priority order for 2 x Tea at Rs. 15.00 = Rs. 30.00; the 10% priority charge of Rs. 3.00 gives Rs. 33.00 payable, and the order goes to Serving Station 1.
- Fig. 5: stations 1 to 4 are busy with tokens 101 to 104, so the priority token 106 and the normal token 107 wait in separate FIFO queues; the cancelled token 105 is no longer in any queue.
- Fig. 6: when a station becomes free, the priority token 106 is served first even though the normal token 105 arrived earlier.
- Fig. 7: three completed orders (Rs. 20.00, Rs. 44.00 and Rs. 75.00) are listed in ascending order of bill amount.

Conclusion / learning Key outcome or learning:

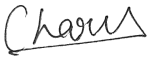
Choosing a suitable data structure for each sub-problem made the canteen workflow efficient: linked queues for fair priority scheduling, a hash table for fast token lookup, and files for persistence. We also learned to combine OOP design with a timer thread and mutex to keep shared data consistent, and to divide a larger program into modules that each member could build and test.

Student signatures

1. Arnav Sankapal 

2. Arnav Patidar 

3. Ayaan Shaikh 

4. Charu Singla 

Faculty remarks: